

PromptGate business and technical tradeoffs

Student: Siwen Liu | Course: PE6201 Emerging AI Technologies | Project: PromptGate

Problem and outcome

I designed PromptGate to help an enterprise AI engineer screen hostile inputs before they reach a RAG assistant. My conclusion is that the prototype is useful for evaluating a screening policy, but its external false-positive rate is too high for production. The primary persona is Mei, an engineer releasing an assistant who understands model metrics but cannot inspect every request. She can use the console to distinguish Allow, Block and Review, instead of treating a fluent model explanation as permission. No organisational time savings or prevented incidents have been measured.

The closest tool identified in my proposal was Protect AI LLM Guard. My contribution is a small, inspectable experiment with frozen policies, cross-dataset evaluation and explicit review costs, not a claim to outperform that product. The scope is one standalone English input. Retrieved documents, conversation history, model outputs, user identity and downstream tool execution remain outside this prototype.

Design and build or buy decisions

I chose narrow supervised classification with deterministic routing. A regex baseline offers simplicity but recognises only fixed patterns. Labelled text supports a broader classifier without requiring an autonomous agent. The initial MVP was one prompt, one GPT-4o-mini structured-output call and one validated result. The final extension adds local classification, probability calibration, Review routing and a FastAPI console. An optional LLM explains Review only; code rejects explanation requests for other actions and never lets the explanation change the decision.

I own the serving interface, orchestration, policy, data preparation and evaluation logic, using existing Python libraries rather than implementing machine learning from scratch. These layers determine what is forwarded and how performance is measured. I rent GPT-4o-mini through OpenRouter for the baseline and optional explanations. I exclude retrieval because the gateway does not answer questions, and skip low-code to retain control over split isolation, thresholds and failure tests. The local classifier avoids a per-request API fee, although hosting, maintenance and human review are not free. The 100-call baseline cost US\$0.0066129, approximately US\$0.000066 per call, with 3.42-second p95 latency. This measured cost does not establish the cost of Review explanations or a production service.

Data and experimental choices

Following the instructor's feedback, the Naive Bayes and logistic regression candidates were fitted on the original 11,089-row S-Labs training CSV. Labels distinguish benign text from injection. Of 2,101 validation rows, 116 overlapping test texts were removed during preparation, then four overlaps with the original training CSV were excluded. The remaining 1,981 rows became 990 calibration rows and 991 policy-selection rows. Preserving the original training set also preserves its duplicates and conflicting labels; the audit records this limitation rather than describing the training data as fully cleaned.

I used sigmoid calibration, then selected the binary threshold for maximum development recall subject to at most 5% benign false positives. A separate margin search selected Review routing under a 19% development review budget. Both candidates use word TF-IDF; logistic regression additionally uses character features. The selected candidate, Naive Bayes, had higher policy-split recall. External performance did not determine that selection.

The 662-row deepset benchmark contains 263 attacks and 399 benign inputs. Its licence metadata conflicts, so raw text remains local and evaluation-only. S-Labs source files, split documentation and text-free prediction records accompany the repository; deepset acquisition instructions and checksums preserve reproducibility without redistributing its text. Data preparation inspected labels for integrity checks, and public datasets were explored during the proposal. This is a frozen evaluation protocol, not a claim that nobody had previously seen the benchmark.

Results and evaluation critique

My final target was at least 80% attack block recall with no more than 5% benign block FPR on external data. The historical 58.3% Naive Bayes figure in the proposal is not my measured baseline. On deepset, the measured regex baseline achieved 1.9% recall and 0% FPR; always predicting benign would achieve 60.3% accuracy. Accuracy alone therefore hides the security failure.

The fitted Naive Bayes binary comparison achieved 99.6% recall and 4.0% FPR on the policy split, but 79.8% recall and 43.1% FPR externally. Logistic regression reached 69.2% recall and 16.0% FPR externally. Neither met the joint target. I retain the development-selected model instead of choosing whichever looks best after inspecting external labels.

The actual hybrid web policy blocked 201 of 263 attacks, giving 76.4% recall, and falsely blocked 147 of 399 benign inputs, giving 36.8% FPR. It routed 185 of 662 inputs to Review, or 27.9%, above the development budget. Review captured 53 of 225 counterfactual binary errors, or 23.6%; it did not resolve those errors. Including Review, 295 of 399 benign inputs were held, or 73.9%. That operational burden makes automatic deployment unjustified even though blocking FPR fell. Twenty-five attacks were still allowed.

The separate 100-call LLM baseline achieved 50.0% block recall, 0% FPR and 13% Review with no technical failures. It missed the proposal's 58.3% MVP target. The sample has only 50 benign cases, so zero observed false positives is not proof of zero underlying risk. This run and the 662-row classifier benchmark use different samples; they cannot establish a fair model ranking. Neither experiment measures successful compromise of a downstream assistant or human review accuracy. The large cross-dataset gap is consistent with distribution shift, but does not isolate whether wording, labels or attack composition caused it. Scores calibrated on S-Labs should not be interpreted as reliable external probabilities.

Engineering difficulties and responsible use

The implementation addresses concrete failure cases: overlapping validation examples are removed; malformed or conflicting LLM outputs become Review; missing credentials make no API call; and invalid explanation responses leave the Review decision unchanged. Regression tests exercise these behaviours. These controls make failures visible, but cannot prevent a confidently wrong Allow. Versioned benchmark evidence helps detect regressions; deployment would additionally require sampled human audits and drift monitoring, which are not implemented.

I use OWASP Top 10 for LLM Applications 2025 as an AI-security framing, not a certification. False positives require an agreed review capacity; false negatives require downstream safeguards beyond this gateway. Provider retention remains outside local control, so optional calls should use public or synthetic text. The console has no persistent approval queue, authentication or deployed RAG connection. Its May forward flag is advisory, not proof that downstream enforcement exists.

Next decision

I would keep PromptGate as a research MVP. Before deployment, I would collect organisation-specific labels, agree on false-positive and review costs, retune only on a new development set, and evaluate once on a separate holdout. This project demonstrates a running gateway and an auditable evaluation, while showing why a strong development score is insufficient evidence for a production security decision.

Evidence and sources

Measured results: repository evidence/offline/summary.json and evidence/live/summary.json, with corresponding prediction records. Dataset attribution and pinned source links: data_sources/README.md. Method and metric definitions: evals/README.md. Security framing: OWASP Top 10 for LLM Applications 2025. Product comparator: Protect AI LLM Guard, identified in the original proposal, not benchmarked here.